

# K-CUBE Technical Foundation Review

Prepared: 2026-05-30

## Review Scope

This review responds to the requirement-analysis and infrastructure-audit phase. It focuses on structural foundation, not UI screen design.

Reviewed local project areas:

- database/schema.sql
- backend/src/app.ts
- backend/src/routes/\*
- backend/src/controllers/\*
- frontend/src/app/\*
- frontend/src/lib/api.ts
- Existing docs in docs/

The supplied Google Sheet URL was noted as stakeholder context, but the local implementation should treat database/schema.sql and these dated documents as the technical baseline.

## Current Platform Foundation

| Layer    | Status          | Notes                                                                                                                          |
|----------|-----------------|--------------------------------------------------------------------------------------------------------------------------------|
| Frontend | Started         | Next.js App Router pages exist for home, auth, dashboard, events, learning, K-Food, rewards, admin, and content detail routes. |
| Backend  | Started         | Express/TypeScript API with route modules for auth, users, admin, analytics, engagement, map, and search.                      |
| Database | Started         | MySQL schema covers users, points, referrals, learning, uploads, K-Food, CMS, search, analytics, and admin audit.              |
| Docker   | Started         | Compose can launch frontend, backend, and MySQL together. Backend Docker MySQL host is now overridden to mysql .               |
| Auth     | Foundation only | Email/password, OTP placeholder, JWT, refresh token, and Google placeholder exist. Google SSO needs production verification.   |
| Events   | Partial         | Frontend event pages exist; first-class event tables/API and Google Calendar sync are still required.                          |

## Attached Code Foundation Review

### App.js

The attached App.js is useful as a route/menu reference, but the current frontend is Next.js, not React Router. Integration should happen by mapping route intent into frontend/src/app pages and shared components.

Do not add a second router layer unless the project intentionally leaves Next.js routing.

## User.js

The attached `User.js` Mongoose schema is useful as a user-domain reference, but the backend currently uses MySQL, not MongoDB. Integration should happen through:

- `users`
- `auth_identities`
- `point_transactions`
- `referrals`
- `session_logs`

Do not introduce MongoDB/Mongoose unless the architecture is formally changed.

## Required Foundation Work Before UI Screens

1. Upgrade Google SSO from placeholder to verified Google Identity Services/OIDC flow. 2. Add first-class event tables:

- `platform_events`
- `platform_event_rsvps`
- `google_calendar_connections`
- `google_calendar_sync_jobs`

3. Add event APIs and admin event management APIs. 4. Add Google Calendar sync service. 5. Standardize response shape across all backend endpoints. 6. Harden auth and admin security:

- refresh token storage
- OTP production provider
- admin field allowlists
- audit logs
- CORS allowlist

7. Add transaction boundaries around all point awards.

## Proposed Backend Module Structure

```
backend/src/  
  controllers/  
    authController.ts  
    eventController.ts  
    googleWorkspaceController.ts  
  services/  
    googleAuthService.ts  
    googleCalendarService.ts  
    eventService.ts  
    pointsService.ts  
  routes/  
    auth.ts  
    events.ts  
    googleWorkspace.ts  
  middleware/  
    auth.ts  
    rateLimiter.ts
```

## Proposed Frontend Module Structure

```
frontend/src/
  app/
    events/
      page.tsx
      [slug]/page.tsx
    signin/page.tsx
    dashboard/page.tsx
    admin/page.tsx
  lib/
    api.ts
    googleAuth.ts
  components/
    auth/
      GoogleSignInButton.tsx
    events/
      EventList.tsx
      EventDetail.tsx
      EventRsvpButton.tsx
```

## Infrastructure Notes

Docker local URLs:

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:4000/api/v1`
- MySQL: `localhost:3306`

For Docker networking, backend must use:

```
MYSQL_HOST=mysql
```

For local non-Docker development, backend can use:

```
MYSQL_HOST=127.0.0.1
```

## Deliverables Created Today

| Document                                                          | Purpose                                                              |
|-------------------------------------------------------------------|----------------------------------------------------------------------|
| <code>docs/2026-05-30/DATABASE_ERD_AND_SCHEMA.md</code>           | ERD, table relationships, Google Calendar tables, User.js mapping    |
| <code>docs/2026-05-30/API_SPECIFICATION.md</code>                 | API contracts for frontend/backend, Google SSO, Google Calendar sync |
| <code>docs/2026-05-30/GOOGLE_WORKSPACE_INTEGRATION_PLAN.md</code> | Implementation plan for Google SSO and Calendar API                  |
| <code>docs/2026-05-30/TECHNICAL_FOUNDATION_REVIEW.md</code>       | Requirement-analysis and infrastructure-audit review                 |

## Decision Summary

- Continue with MySQL, not MongoDB/Mongoose.

- Continue with Next.js App Router, not React Router.
- Use Google Identity Services/OIDC for SSO.
- Use K-CUBE database as source of truth for events.
- Sync K-CUBE events outward to Google Calendar.
- Add Workspace domain-wide delegation only if organization-wide calendar access is required.